MI Evolutionary Algorithms - N királynő probléma

(Makai Marcell – D53G1U)

1 Tartalomjegyzék

Tartalomjegyzék

1 TARTALOMJEGYZÉK ... 2

2 BEVEZETÉS ... 3

3 A FELADAT LEÍRÁSA ... 3

3.1 KIRÁLYNŐ MOZGÁSA ... 3

4 ELMÉLETI HÁTTÉR ... 4

4.1 EVOLÚCIÓS ALGORITMUSOK ... 4

4.2 GENETIKUS ALGORITMUSOK ... 4

5 MEGOLDÁS PSZEUDOKÓDJA ... 5

6 PARAMÉTEREK ... 6

6.1 ALKALMAZOTT PARAMÉTEREK ... 6

6.2 PARAMÉTERHANGOLÁS ... 6

6.2.1 Mutációs ráta vizsgálata ... 6

6.2.2 Populációméret vizsgálata ... 6

6.2.3 Elitizmus vizsgálata ... 7

6.2.4 Végleges paraméterválasztás ... 7

6.3 MEGFIGYELÉSEK ... 7

6.3.1 A nagyobb populáció segített ... 7

6.3.2 A magasabb mutációs ráta jobb lett ... 7

6.3.3 Az elitizmus nem mindig javít a találatokon ... 8

7 PROGRAM FUTTATÁSA ... 8

7.1 ALAP PROGRAM FUTTATÁSA FIX ÉRTÉKEKKEL ... 8

7.2 BENCHMARK FUTTATÁSA ... 8

8 EREDMÉNYEK ÖSSZEHASONLÍTÁSA KÜLÖNBÖZŐ N ÉRTÉKEKKEL ... 8

8.1 N ÉRTÉKENKÉNT LEGJOBB EREDMÉNY ... 8

9 HIVATKOZÁSOK ... 9

2 Bevezetés

Az optimalizálási problémák megoldása a mesterséges intelligencia egyik meglehetősen fontos területe. Az ilyen problémák esetében gyakran nem létezik determinisztikus, hatékony megoldás, ezért heurisztikus és metaheurisztikus módszereket alkalmazunk. Az evolúciós algoritmusok ezen módszerek közé tartoznak, és a természetes szelekció elvén alapulnak [1].

A genetikus algoritmusok az evolúciós algoritmusok egyik legismertebb csoportját alkotják, amelyek populáció alapú keresést alkalmaznak, és olyan operátorokat használnak, mint a szelekció, keresztezés és mutáció [2].

3 A feladat leírása

A N királynő probléma célja, hogy egy N×N-es sakktáblán úgy helyezzünk el N darab királynőt, hogy azok ne üssék egymást.

Ez a probléma klasszikus példája a megszorítási kielégítési problémáknak (Constraint Satisfaction Problem, CSP), amelyet széles körben tárgyal a szakirodalom [3].

3.1 Királynő mozgása

Az alábbi irányokban tud lépni a sakktáblán:

- vízszintesen,
- függőlegesen,
- átlósan.

Ezért egy helyes megoldásban:

- nincs két királynő ugyanabban a sorban,
- nincs két királynő ugyanabban az oszlopban,
- nincs két királynő ugyanazon az átlón.

4 Elméleti háttér

4.1 Evolúciós algoritmusok

Az evolúciós algoritmusok olyan optimalizációs módszerek, amelyek a természetes evolúció folyamatát modellezik. Egy populációból indulnak ki, amely egyedekből (lehetséges megoldásokból) áll, és iteratív módon javítják a megoldásokat [1].

A főbb lépések:

- populáció inicializálása,
- fitness érték számítása,
- szelekció,
- keresztezés,
- mutáció.

4.2 Genetikus algoritmusok

A genetikus algoritmusok az evolúciós algoritmusok egy speciális típusa, amelyben az egyedek genetikai reprezentációval rendelkeznek. A populáció fejlődése során a jobb fitness értékű egyedek nagyobb eséllyel kerülnek kiválasztásra [2].

A legfontosabb operátorok:

- Szelekció: jobb egyedek kiválasztása (pl. tournament selection),
- Keresztezés: két szülő kombinálása,
- Mutáció: véletlenszerű változtatás az egyedeken.

5 Megoldás pszeudokódja

```
NKiralynoProblema()

    Inicializálj egy P populációt véletlen permutációkkal
    Számítsd ki minden egyed fitness értékét

    ciklus amíg nem teljesül a leállási feltétel:
        ÚjPopuláció = legjobb egyedek (elitizmus)

        amíg ÚjPopuláció mérete kisebb, mint a kívánt populációméret:
            szülő1 = tournament_szelekció(P)
            szülő2 = tournament_szelekció(P)

            valószínűséggel pc:
                gyermek1, gyermek2 = keresztezés(szülő1, szülő2)
            különben:
                gyermek1 = szülő1 másolata
                gyermek2 = szülő2 másolata

            valószínűséggel pm:
                mutáció(gyermek1)

            valószínűséggel pm:
                mutáció(gyermek2)

            add hozzá gyermek1-et és gyermek2-t ÚjPopulációhoz

        P = ÚjPopuláció
        Számítsd ki az új populáció fitness értékeit

        ha van olyan egyed, amelynek fitness értéke N * (N - 1) / 2:
            állj le

    add vissza a legjobb megoldást
```

6 Paraméterek

6.1 Alkalmazott paraméterek

Az alábbi paramétereket lehet beállítani a genetikus algoritmus futtatásához:

Paraméter	Érték	Jelentés
population_size	200	Egy generációban szereplő egyedek száma
generations	300	Maximális generációszám
crossover_rate	0.95	Keresztezés valószínűsége
mutation_rate	0.5	Mutáció valószínűsége
elite_size	2	Változatlanul továbbvitt legjobb egyedek száma
tournament_size	4	Tournament selection során kiválasztott egyedek száma
runs	30	Futtatások száma
N	8, 10, 12, 14	Vizsgált sakktáblaméretek

A fitness függvény a nem ütköző királynők számát maximalizálja. A megoldás akkor tekinthető sikeresnek, ha a konfliktusok száma 0.

6.2 Paraméterhangolás

A paraméterhangolás lényege, hogy megtaláljuk, hogy a genetikus algoritmus milyen értékekkel talál stabilan és gyorsan megoldást. A kísérletek során többféle mutációs ráta, populációméret és elitizmus érték került kipróbálásra.

A paramétereket a benchmark alapján választottam ki, mellyel végigteszteltem a vizsgált értékekkel elérhető minden kombinációt.

6.2.1 Mutációs ráta vizsgálata

Mutációs ráta	Tapasztalat
0.05	Alacsony sikerességet adott nagyobb N értékeknél, gyakran beragadt lokális optimumba.
0.10	Kisebb N esetén elfogadható volt, de N=12 és N=14 esetén már kevésbé teljesített jól.
0.30	Jó eredményeket adott, de nagyobb N értékeknél nem volt minden esetben a legerősebb.
0.50	A mérések alapján nagyobb N értékeknél ez adta a legjobb sikerességi arányt.

A végleges értéknek *0.50* lett kiválasztva, mert ez biztosította a legjobb sikerességi arányt.

6.2.2 Populációméret vizsgálata

Populációméret	Tapasztalat
50	Gyorsabb lehet, de nagyobb N értékeknél jelentősen csökkent a sikeresség.

100	Közepes méretű problémáknál elfogadható eredményt adott, de nem volt minden esetben elég stabil.
200	A legstabilabb eredményeket adta, különösen N=10, N=12 és N=14 esetén.

A végleges érték *200* lett, mivel ez biztosította a legjobb stabilitást és a legmagasabb sikerességi arányt a vizsgált N értékek többségénél.

6.2.3 Elitizmus vizsgálata

Elite size	Tapasztalat
0	Bizonyos esetekben jó eredményt adott, de kevésbé stabil, mert a legjobb egyedek elveszhetnek.
2	Jó kompromisszumot adott: megőrzi a legjobb egyedeket, de nem csökkenti túlzottan a változatosságot.
4	Több esetben megfelelő volt, de nagyobb N értékeknél nem mindig javította az eredményeket.
10	Túl erős elitizmusnak bizonyult, csökkenthette a populáció diverzitását.

A végleges érték *2* lett, mert ez adta a legjobb kompromisszumot a stabilitás és a populáció változatosságának fenntartása között.

6.2.4 Végleges paraméterválasztás

Az alkalmazott paramétereim az 5.1-es pontban megtalálhatóak, azért választottam végül ezeket, mivel így az algoritmus kis méretű problémákon gyorsan, nagyobb N értékek esetén pedig elfogadható sikerességgel talált megoldást. A magas keresztezési ráta segíti a jó részmegoldások kombinálását, a 0.50-es mutációs ráta pedig nagyobb változatosságot biztosít a populációban, így csökkenti a lokális optimumba ragadás esélyét.

6.3 Megfigyelések

6.3.1 A nagyobb populáció segített

A *200*-as méret általában jobb volt, főleg *N=10, 12, 14* esetében. Ez jól mutatja, hogy a nagyobb populáció növeli a keresési tér lefedését.

6.3.2 A magasabb mutációs ráta jobb lett

A *0.5*-ös érték több esetben nagyon jól teljesít, főleg nagyobb N értékeknél. Ez azt jelenti, hogy az algoritmusnak szüksége van erősebb véletlenszerű változásra, különben könnyebben beragad lokális optimumokba.

6.3.3 Az elitizmus nem mindig javít a találatokon

A túl magas elitizmus csökkentheti a populáció diverzitását, ezért nagyobb N értékek esetében nem minden esetben javította az eredményeket.

7 Program futtatása

A programot a **main.py** indításával lehet lefuttatni. Az alábbi parancs segítségével lehet futtatni docker segítségével a projekt mappájában futtatva:

7.1 Alap program futtatása fix értékekkel

„docker run --rm -v $(pwd):/app -w /app python:3.9 python main.py”

7.2 Benchmark futtatása

„docker run --rm -v $(pwd):/app -w /app python:3.9 python benchmark.py”

Ebben az esetben az eredmény a *benchmark_results.csv* fájlba kerül a futás végén.

8 Eredmények összehasonlítása különböző N értékekkel

N	Sikeres futás	Átlagos generációszám
8	30/30	1.23
10	30/30	12.33
12	30/30	16.13
14	28/30	47.82

A végleges paraméterek kiválasztása után az algoritmust minden N értékre 30 alkalommal futtattam le különböző seedekkel. A táblázat a sikeres futások számát és sikeres futásokhoz tartozó átlagos generációszámot mutatja.

8.1 N értékenként legjobb eredmény

N	Legjobb paraméter	Sikeres futás	Átlagos generációszám
8	pop=200, mut=0.3, elite=2	30/30	1.23
10	pop=200, mut=0.1, elite=0	30/30	4.8
12	pop=200, mut=0.5, elite=2	30/30	16.13
14	pop=100, mut=0.5, elite=0	30/30	88.60

9 Hivatkozások

[1] T. Bäck, D. B. Fogel, and Z. Michalewicz, Evolutionary Computation 1: Basic Algorithms and Operators," Bristol, U.K.: Institute of Physics Publishing, 2000.

[2] D. E. Goldberg, Genetic Algorithms in Search, Optimization, and Machine Learning," Boston, MA, USA: Addison-Wesley, 1989.

[3] S. J. Russell and P. Norvig, "Artificial Intelligence: A Modern Approach," 3rd ed., Upper Saddle River, NJ, USA: Prentice Hall, 2010.